ИТМО

Безопасность и данные в Spring

Аутентификация, Spring Security, TLS и HTTPS,
Spring Data JPA и проблема N+1

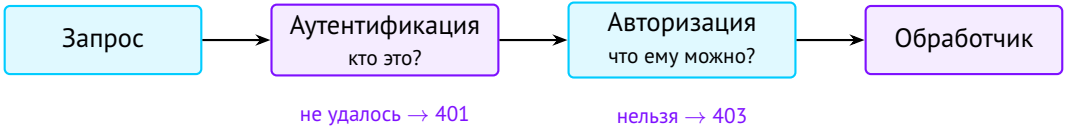
Санкт-Петербург, 13 октября 2026 г.

План лекции



Сервер ничему не верит на слово: личность и права проверяются в **каждом** запросе.
ORM пишет SQL за вас — и его нужно уметь читать.

Аутентификация и авторизация



Аутентификация

Проверка личности: пароль, токен, сертификат.

Результат — известный пользователь с набором ролей.

Авторизация

Проверка прав: `ADMIN` может удалять, `USER` — только читать.

Без аутентификации невозможна: сначала «кто», потом «что можно».

HTTP не помнит пользователя

Способ	Что в каждом запросе	Где состояние
Basic	логин и пароль в заголовке `Authorization`	нигде
Токен	токен в заголовке `Authorization: Bearer`	в самом токене

HTTP не хранит состояние между запросами: доказательство личности приходит с **каждым** запросом, и сервер проверяет его **каждый** раз

Basic Authentication

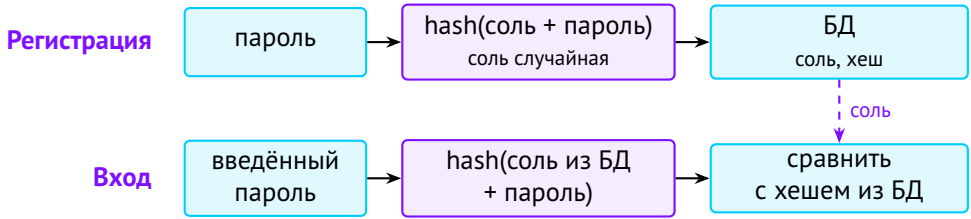
```
GET /students HTTP/1.1
Authorization: Basic aXZhbm92OnF3ZXJ0eQ==

base64("ivanov:qwerty")
  = aXZhbm92OnF3ZXJ0eQ==
```

- Логин и пароль уходят в **каждом** запросе
- base64 — кодирование, а не шифрование: раскодируется одной командой
- Без HTTPS пароль виден любому на пути запроса
- Нет выхода из системы: браузер запоминает пароль сам

Годится для служебных API и тестов поверх HTTPS. Для пользователей — токены

Пароли не хранят — хранят хеш



Хеш — односторонняя функция: из хеша пароль не восстановить. При утечке БД пароли не раскрываются сразу.

Соль — случайная добавка: одинаковые пароли дают разные хеши, заранее посчитанные таблицы хешей бесполезны.

Почему не SHA-256

ИТМО

Быстрый хеш — плохо

SHA-256 и MD5 созданы, чтобы считаться **быстро**. Одна видеокарта перебирает миллиарды вариантов в секунду.

Для паролей нужен **намеренно медленный** хеш с настраиваемой стоимостью.

Алгоритм	Чем замедляет
bcrypt	параметр стоимости: каждая единица удваивает время
PBKDF2	число итераций
scrypt	время и **память**
Argon2id	время и память, современный выбор

Медленный хеш замедляет и злоумышленника, и вход пользователя: для одного входа это доли секунды, для перебора — годы

Регистрация и вход: алгоритм

```
register(login, password):
  salt = random_bytes(16)
  hash = slow_hash(password, salt, cost)
  db.save(login, salt, cost, hash)

login(login, password):
  user = db.find(login)
  if user is none:
    fail("wrong login or password")
  h = slow_hash(password, user.salt,
                user.cost)
  if not equal_const_time(h, user.hash):
    fail("wrong login or password")
  issue_token(user)
```

- `slow_hash` — bcrypt или Argon2id, `cost` — насколько медленно
- Соль и стоимость хранятся рядом с хешем: при входе хеш пересчитывается с **теми же** параметрами
- Одна ошибка на оба случая: не раскрывать, какие логины существуют
- Сравнение за постоянное время: по времени ответа не угадать, сколько символов совпало
- В Spring это `PasswordEncoder`: `encode` и `matches`

Как устроен JWT

ИТМО

header
`{alg: RS256}`

payload
`{sub: ivanov, roles: [USER], exp: 1767225600}`

signature
подпись header + payload

каждая часть в base64url, между ними точки

Открыт

Токен не зашифрован: payload читает кто угодно — секреты туда не кладут. Подпись гарантирует, что токен **не подделан**.

Подпись

HS256 — общий секрет.
RS256 — подпись приватным ключом, проверка публичным.

Без состояния

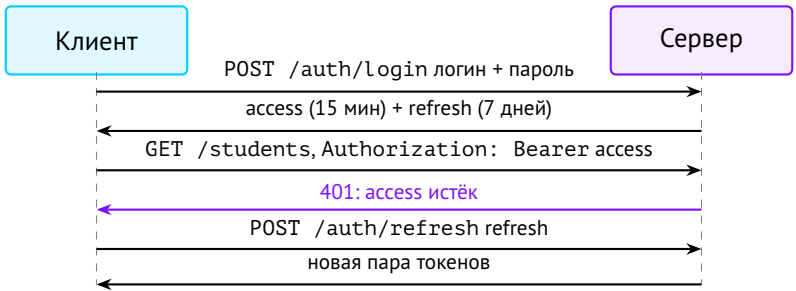
Сервер ничего не хранит: проверяет подпись и exp. Минус — выданный токен не отозвать до истечения.

Что проверять в JWT

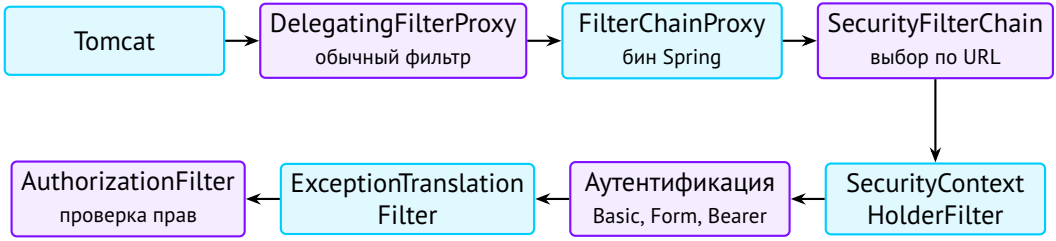
Проверка	Зачем
подпись	без неё любой может вписать себе `roles: [ADMIN]`
алгоритм	ожидаемый алгоритм задаёт сервер, а не заголовок токена: атаки `alg: none` и подмена RS256 на HS256
`exp`, `nbf`	токен не просрочен и уже действует
`iss`	токен выдал доверенный сервер
`aud`	токен выдан именно для этого API

Свою проверку JWT не пишут: библиотеки и фреймворки делают её правильно

Access- и refresh-токены

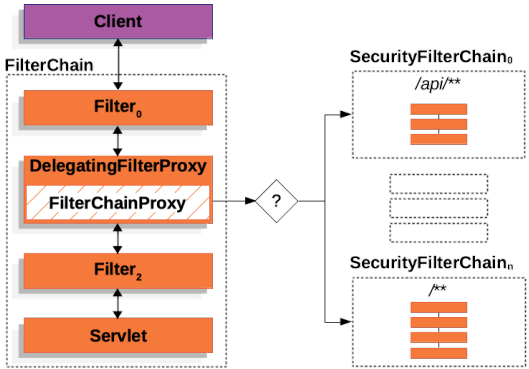


Spring Security: цепочка фильтров



Spring Security целиком живёт в **servlet-фильтрах**, до Spring MVC. `DelegatingFilterProxy` — мост: Tomcat видит обычный фильтр, а работу делает бин из контекста. В цепочке около 15 фильтров, их порядок фиксирован.

Архитектура Spring Security

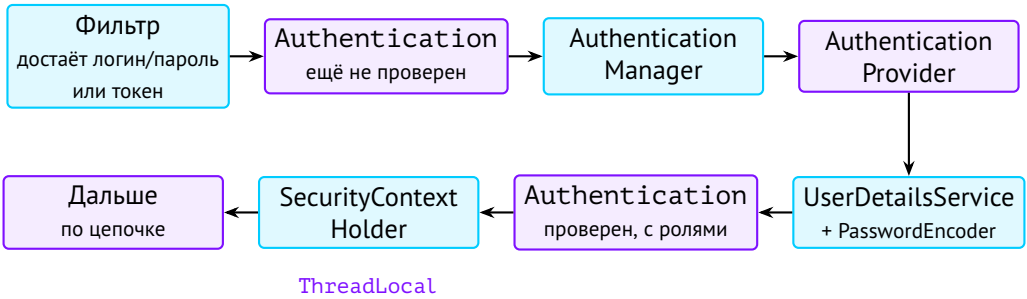


Spring Security Reference

- Tomcat прогоняет запрос через обычную цепочку фильтров; один из них — `DelegatingFilterProxy`
- Он отдаёт запрос бину `FilterChainProxy`
- `FilterChainProxy` выбирает `SecurityFilterChain` по URL и прогоняет запрос через её фильтры
- Цепочек может быть несколько, срабатывает первая подходящая: `/api/**` — JWT, `/**` — форма входа

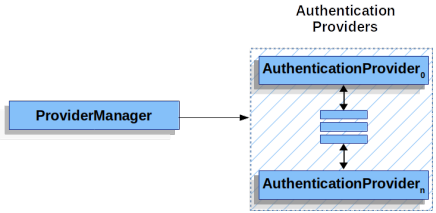
https://docs.spring.io/spring-security/reference/servlet/architecture.html

Аутентификация изнутри



401: `AuthenticationException` — кто делает запрос, неизвестно.
403: `AccessDeniedException` — пользователь известен, но прав не хватает

ProviderManager и провайдеры



Spring Security Reference

```
public interface AuthenticationProvider {
  Authentication authenticate(Authentication a)
      throws AuthenticationException;
  boolean supports(Class<?> authentication);
}
```

`AuthenticationManager` — один метод `authenticate`. Его реализация `ProviderManager` идёт по списку провайдеров:

- `supports` = `false` — пропустить
- вернул `Authentication` — готово
- вернул `null` или неверный пароль — следующий
- аккаунт заблокирован или отключён — отказ сразу
- никто не справился — родительский менеджер, иначе исключение

Пароль проверяет `DaoAuthenticationProvider`, JWT — `JwtAuthenticationProvider`.

Javadoc: `docs.spring.io/spring-security/reference/api/java/.../ProviderManager.html`

UserDetails

```
public interface UserDetails extends Serializable {
  String getUsername();
  String getPassword();                       // hash
  Collection<? extends GrantedAuthority> getAuthorities();

  // default methods, all return true
  boolean isAccountNonExpired();
  boolean isAccountNonLocked();
  boolean isCredentialsNonExpired();
  boolean isEnabled();
}

public interface UserDetailsService {
  UserDetails loadUserByUsername(String username)
      throws UsernameNotFoundException;
}
```

- Всё, что Spring Security знает о пользователе: логин, хеш пароля, права
- Флаги проверяет `DaoAuthenticationProvider`: заблокирован — `LockedException`, отключён — `DisabledException`
- Готовая реализация — `User.withUsername(...)` или своя сущность
- После входа лежит в `getPrincipal()` у `Authentication`

Javadoc: `docs.spring.io/spring-security/reference/api/java/.../UserDetails.html`

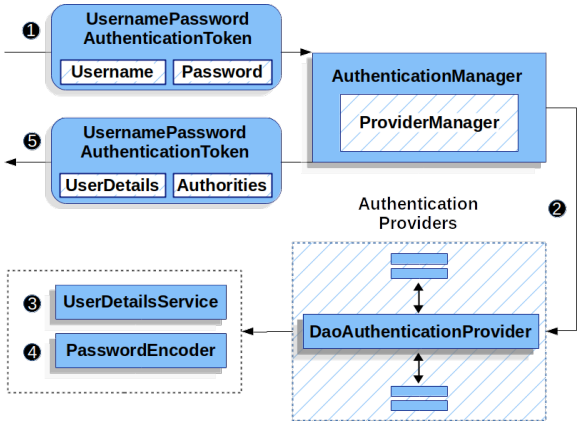
Пользователи из базы

```
@Bean
PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder();
}

@Service
public class DbUserDetailsService
    implements UserDetailsService {
  private final UserRepository users;   // via constructor
  @Override
  public UserDetails loadUserByUsername(String name) {
    AppUser u = users.findByUsername(name)
        .orElseThrow(() ->
            new UsernameNotFoundException(name));
    return User.withUsername(u.getUsername())
        .password(u.getPasswordHash())  // bcrypt
        .roles(u.getRole())             // ROLE_ADMIN
        .build();
  }
}
```

- Достаточно двух бинов: Spring сам соберёт `DaoAuthenticationProvider`
- Сервис только **находит** пользователя, пароль сравнивает провайдер через `PasswordEncoder`
- «Нет такого пользователя» и «неверный пароль» снаружи выглядят одинаково — `BadCredentialsException`
- `roles("ADMIN")` хранится как authority `ROLE_ADMIN`

Вход по паролю: вся схема



1. Фильтр упаковывает логин и пароль в `UsernamePassword-AuthenticationToken`
2. `ProviderManager` отдаёт его провайдеру `DaoAuthenticationProvider`
3. Провайдер загружает `UserDetails` через ваш `UserDetailsService`
4. Сравнивает пароль с хешем через `PasswordEncoder`
5. Возвращает токен с `UserDetails` и ролями, фильтр кладёт его в `SecurityContextHolder`

Spring Security Reference

`docs.spring.io/spring-security/reference/.../dao-authentication-provider.html`

AuthenticationManagerBuilder

```
@Bean
SecurityFilterChain security(HttpSecurity http,
    UserDetailsService users, PasswordEncoder encoder,
    ApiKeyAuthenticationProvider apiKeys) throws Exception {
  AuthenticationManagerBuilder builder =
      http.getSharedObject(AuthenticationManagerBuilder.class);
  // DaoAuthenticationProvider
  builder.userDetailsService(users).passwordEncoder(encoder);
  builder.authenticationProvider(apiKeys);     // own provider
  // ProviderManager
  AuthenticationManager manager = builder.build();
  return http
      .authenticationManager(manager)
      .authorizeHttpRequests(a ->
          a.anyRequest().authenticated())
      .httpBasic(Customizer.withDefaults())
      .build();
}
```

- Builder собирает `ProviderManager`: каждый вызов добавляет провайдер, порядок вызовов — порядок опроса
- `build()` — только один раз
- Не настроили — Spring сам соберёт глобальный менеджер из бинов `UserDetailsService` и `PasswordEncoder`
- Он же для своего `/auth/login`: `AuthenticationConfiguration.getAuthenticationManager()`

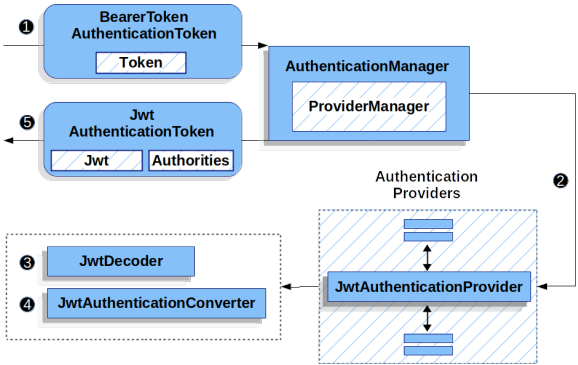
Javadoc: `docs.spring.io/spring-security/reference/api/java/.../AuthenticationManagerBuilder.html`

Stateless API с JWT

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
  return http
      .csrf(c -> c.disable())                                        // token in header, no CSRF
      .sessionManagement(s -> s.sessionCreationPolicy(STATELESS)) // no HttpSession
      .authorizeHttpRequests(a -> a
          .requestMatchers("/auth/**").permitAll()
          .requestMatchers(GET, "/students/**").hasAnyRole("USER", "ADMIN")
          .anyRequest().hasRole("ADMIN"))
      .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(roles())))
      .build();
}

JwtAuthenticationConverter roles() {
  var authorities = new JwtGrantedAuthoritiesConverter();
  authorities.setAuthoritiesClaimName("roles");   // claim with roles
  authorities.setAuthorityPrefix("ROLE_");         // hasRole("ADMIN") checks ROLE_ADMIN
  var converter = new JwtAuthenticationConverter();
  converter.setJwtGrantedAuthoritiesConverter(authorities);
  return converter;
}
```

Проверка JWT: та же схема



Spring Security Reference

https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

1. Фильтр достаёт токен из заголовка `Authorization: Bearer`
2. Тот же `ProviderManager`, другой провайдер — `JwtAuthenticationProvider`
3. `JwtDecoder` проверяет подпись, `exp`, `iss` и разбирает claims
4. `JwtAuthenticationConverter` превращает claims в роли — это наш `roles()`
5. `JwtAuthenticationToken` уходит в `SecurityContextHolder`

Выдача токенов

```
@PostMapping("/auth/login")
TokenPair login(@RequestBody LoginDto r) {
  Authentication a = authManager.authenticate(
      new UsernamePasswordAuthenticationToken(
          r.username(), r.password()));
  return tokens.issue(a);
}

@PostMapping("/auth/refresh")
TokenPair refresh(@RequestBody RefreshDto r) {
  RefreshToken old = refreshRepo
      .findValid(r.token())
      .orElseThrow(Unauthorized::new);
  refreshRepo.revoke(old);     // rotation
  return tokens.issue(old.getUser());
}
```

```
String access(String user, List<String> roles) {
  Instant now = Instant.now();
  JwtClaimsSet claims = JwtClaimsSet.builder()
      .issuer("students-app")
      .subject(user)
      .issuedAt(now)
      .expiresAt(now.plus(15, MINUTES))
      .claim("roles", roles)
      .build();
  return encoder
      .encode(JwtEncoderParameters.from(claims))
      .getTokenValue();
}
```

Refresh — случайная строка, в БД хранится её хеш.

401 и 403 в едином формате

```
http.exceptionHandling(e -> e
    .authenticationEntryPoint((request, response, ex) ->
        problem(response, 401, "Authentication required"))
    .accessDeniedHandler((request, response, ex) ->
        problem(response, 403, "Access denied")));
```

Почему отдельно

Ошибки безопасности возникают в фильтрах, **до** `DispatcherServlet`. `@RestControllerAdvice` их не видит.

Что сделать

`AuthenticationEntryPoint` для 401 и `AccessDeniedHandler` для 403 пишут тот же JSON, что и остальные ошибки API.

Защита методов

```
@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {}

@Service
public class StudentService {

  @PreAuthorize("hasRole('ADMIN')")
  public void delete(long id) { ... }

  @PreAuthorize(
      "hasRole('ADMIN') or #username == authentication.name")
  public Profile profile(String username) { ... }
}
```

- Выражения на SpEL: роли, параметры метода, текущий пользователь
- Работает через **AOP-прокси**: вызов из того же класса проверку обходит
- `hasRole('ADMIN')` проверяет authority `ROLE_ADMIN`
- Правила для URL и для методов дополняют друг друга

CSRF: запрос от имени пользователя



Защита при cookie: `SameSite=Lax` и CSRF-токен в форме, которого нет у чужого сайта.

Токен в заголовке `Authorization` браузер сам не отправляет — CSRF невозможен. Поэтому в API на JWT CSRF-защиту выключают.

Spring Security включает CSRF-защиту по умолчанию: `POST`, `PUT`, `DELETE` без CSRF-токена получают **403** — в том числе из Postman

Зачем HTTPS



HTTPS = HTTP поверх TLS, порт 443

Шифрование

Трафик зашифрован: пароль, cookie и токены не прочитать на пути запроса — в Wi-Fi кафе, у провайдера.

Целостность

Изменённый по дороге пакет обнаружится: подменить ответ или вставить в страницу рекламу нельзя.

Подлинность

Сервер доказывает, что он и есть `bank.com`. Без этого шифрование бесполезно: можно шифровать прямо злоумышленнику.

Два вида шифрования

Симметричное

Один ключ шифрует и расшифровывает: AES-GCM, ChaCha20. Быстрое — им шифруют все данные.

Проблема: как договориться о ключе, если канал прослушивается?

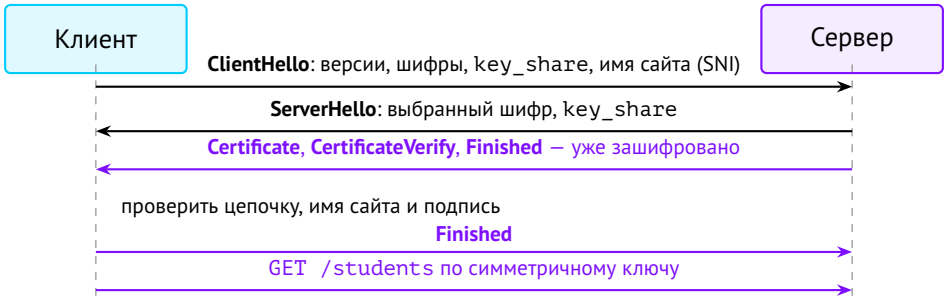
Асимметричное

Пара ключей: **публичный** известен всем, **приватный** — только владельцу.

Подпись приватным ключом проверяется публичным: RSA, ECDSA. Обмен Диффи — Хеллмана (ECDHE): стороны получают общий секрет, не передавая его.

TLS сочетает оба: асимметричная криптография — чтобы договориться о ключе и проверить сервер, симметричная — чтобы шифровать данные

Рукопожатие TLS 1.3



- Из двух `key_share` (ECDHE) стороны вычисляют общий ключ; по сети он не передаётся
- Приватный ключ сервера только **подписывает** рукопожатие: доказывает, что сертификат его
- Ключ новый на каждое соединение: украденный позже приватный ключ не расшифрует старый трафик

Что внутри сертификата

Поле X.509	Пример
Subject, SAN	`itmo.ru`, `*.itmo.ru`
Публичный ключ	ECDSA P-256 или RSA 2048
Issuer	кто выдал (CA)
Срок действия	`notBefore`, `notAfter`
Подпись	полей, ключом издателя

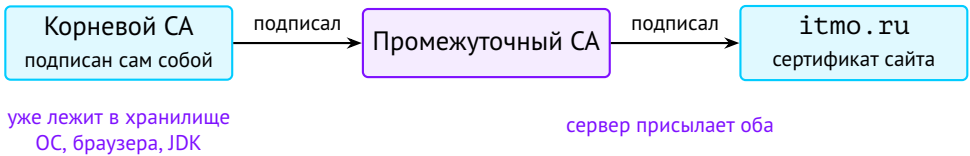
Сертификат

Связывает **имя сайта** и **публичный ключ**; удостоверяющий центр (CA) ручается подписью.

Сертификат открыт, приватный ключ не покидает сервер.

Клиент сверяет имя в SAN с адресом из URL:
сертификат `evil.com` для `bank.com` не подойдёт, даже если он настоящий

Цепочка доверия



Проверка клиентом: подпись каждого звена — ключом следующего вверх, пока не дойдём до корня из своего хранилища. Плюс сроки, имя сайта и отзыв.

Если цепочка не сошлась: браузер показывает «подключение не защищено», Java бросает `PKIX path building failed`.

Откуда берётся сертификат

Let's Encrypt

Бесплатный CA. По протоколу ACME сервер доказывает, что владеет доменом: выкладывает файл или DNS-запись.

Срок — 90 дней, продлевает certbot.

Локально

Самоподписанный: `keytool`, `mkcert`.

Клиенты ему не доверяют, пока его не добавят в хранилище вручную.

TLS на прокси

TLS часто завершается на nginx или балансировщике, приложение получает HTTP и заголовок `X-Forwarded-Proto`.

Keystore — свой сертификат и приватный ключ.
Truststore — чужие сертификаты, которым доверяем

HTTPS в Spring Boot

application.properties

```
server.port=8443
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-type=PKCS12
server.ssl.key-store-password=${KEYSTORE_PASSWORD}
server.ssl.key-alias=app
```

самоподписанный сертификат для разработки

```
keytool -genkeypair -alias app -keyalg EC \
  -storetype PKCS12 -keystore keystore.p12 \
  -validity 365 -dname "CN=localhost" \
  -ext "SAN=dns:localhost"
```

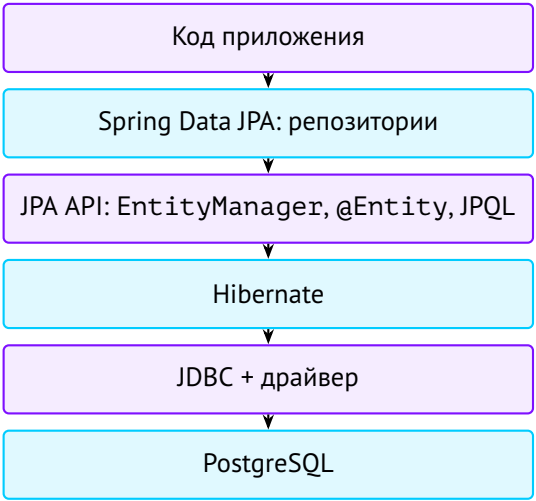
- TLS включает встроенный Tomcat, код контроллеров не меняется
- Пароль хранилища — из переменной окружения, не из git
- За прокси: `server.forward-headers-strategy=native` — приложение узнает, что запрос пришёл по HTTPS
- Spring Security отправляет `Strict-Transport-Security` на HTTPS-ответы: браузер больше не пойдёт на сайт по HTTP

Объекты и таблицы не совпадают

	Объекты	Таблицы
Связь	ссылка: `student.getGroup()`	внешний ключ `group_id`
Коллекции	`List<Student>` в группе	нет, обратная сторона FK
Многие ко многим	две коллекции	промежуточная таблица
Наследование	`extends`	нет, эмулируется
Идентичность	`==` и `equals`	первичный ключ

ORM (object-relational mapping) переводит между двумя моделями, чтобы код работал с объектами, а не со строками результата запроса

JPA, Hibernate и Spring Data



генерирует реализацию
по интерфейсу

стандарт,
пакет `jakarta.persistence`

реализация стандарта,
пишет SQL

Сущность и таблица

```java
@Entity
@Table(name = "students")
public class Student {
  @Id
  @GeneratedValue(strategy = IDENTITY)
  private Long id;

  @Column(name = "last_name", nullable = false)
  private String lastName;

  @Column(nullable = false, unique = true)
  private String email;

  @Enumerated(EnumType.STRING)
  private Status status;

  @Transient private int age;   // not stored

  protected Student() {}         // for JPA
}
```

```sql
create table students (
  id        bigint generated by default
            as identity primary key,
  last_name varchar(255) not null,
  email     varchar(255) not null unique,
  status    varchar(255)
);
```

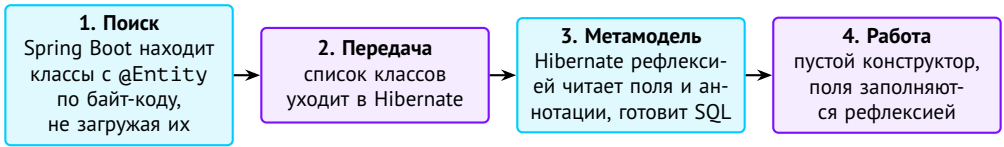
- Такую схему создаст сам Hibernate (`ddl-auto`)
- В production схему ведут миграции (Flyway), Hibernate только сверяет: `validate`

Аннотации сущности

Аннотация	Что задаёт
`@Entity`	класс — сущность; имя таблицы по умолчанию — имя класса
`@Table`	имя таблицы, уникальные ограничения, индексы
`@Id`	первичный ключ
`@GeneratedValue`	кто генерирует ключ: `IDENTITY` — БД при вставке, `SEQUENCE` — последовательность
`@Column`	имя колонки, `nullable`, `unique`, `length`
`@Enumerated`	как хранить enum: `ORDINAL` (номер, по умолчанию) или `STRING`
`@Transient`	поле не хранится в БД
`@Version`	номер версии для оптимистичной блокировки

`ORDINAL`: добавили константу в середину enum — старые строки в БД начали означать другое. Используйте `STRING`

Кто читает @Entity



	`@Autowired`, `@Transactional`	`@Entity`
Кто читает	`BeanPostProcessor` Spring	Hibernate
Когда	при создании каждого бина	один раз при старте
Объект	бин: один, создаёт контейнер	не бин: новый на каждую строку
Результат	внедрение, прокси вокруг бина	таблицы, колонки, готовый SQL

Генерация ключа: IDENTITY и SEQUENCE

ИТМО

IDENTITY

```
repo.save(new Student("Ivanov"));

insert into students
  (email, last_name, status)
values (?, ?, ?)
-- id assigned by the database
```

- `id` известен только после `insert`: Hibernate выполняет его сразу при `save`
- Пакетная вставка (JDBC batch) отключается

SEQUENCE

```
select nextval('student_seq')
-- 1 call per 50 entities (allocationSize)

insert into students
  (email, last_name, status, id)
values (?, ?, ?, ?)
```

- `id` известен до вставки, `insert` уходит при flush пачками
- В Hibernate 6 AUTO = SEQUENCE с последовательностью `student_seq` по имени сущности

Связи: многие к одному

```java
@Entity
public class Student {
  @ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name = "group_id")
  private StudyGroup group;          // owner
}

@Entity
@Table(name = "study_groups")
public class StudyGroup {
  @OneToMany(mappedBy = "group")
  private List<Student> students;    // inverse
}
```

```sql
-- students.group_id
group_id bigint references study_groups (id)
```

- **Владелец связи** — сторона с внешним ключом: `Student.group`
- `mappedBy` — обратная сторона: колонки у неё нет, изменения в `students` в БД не пишутся
- Чтобы перевести студента, меняют `student.setGroup(g)`, а не список группы

Связи: многие ко многим

```
@Entity
public class Student {
  @ManyToMany
  @JoinTable(name = "student_courses",
      joinColumns = @JoinColumn(name = "student_id"),
      inverseJoinColumns = @JoinColumn(name = "course_id"))
  private Set<Course> courses = new HashSet<>();
}
```

```
create table student_courses (
  student_id bigint not null references students,
  course_id  bigint not null references courses,
  primary key (student_id, course_id)
);
```

- Промежуточную таблицу описывает `@JoinTable`, отдельной сущности для неё нет
- `Set`, а не `List`: при изменении `List` Hibernate удаляет все строки связи и вставляет заново
- Нужны свои поля у связи (оценка, дата записи) — делают сущность `Enrollment` и две связи `@ManyToOne`

LAZY и EAGER

Связь	По умолчанию
@ManyToOne	EAGER
@OneToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

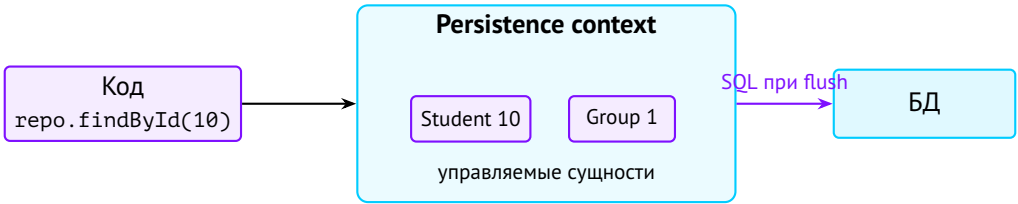
Как работает LAZY

Вместо группы в поле лежит **прокси**. Запрос в БД выполнится при первом обращении к её полям.

EAGER — связанный объект грузится всегда, даже когда не нужен.

Рекомендация: LAZY везде, а нужные связи загружать явно в запросе

Persistence context



- `EntityManager` держит набор **управляемых** сущностей — обычно на время транзакции
- Один `id` — один объект: повторный `findById` вернёт тот же экземпляр без запроса в БД
- Изменения накапливаются и уходят в БД при **flush** — перед коммитом или перед запросом

Dirty checking

```
@Transactional
public void rename(long id, String name) {
  Student s = repo.findById(id).orElseThrow();
  s.setLastName(name);
  // no save() needed
}
```

```
select s1_0.id, s1_0.email, s1_0.group_id,
       s1_0.last_name, s1_0.status
from students s1_0 where s1_0.id = ?

-- commit -> flush:
update students
set email = ?, group_id = ?, last_name = ?, status = ?
where id = ?
```

- При загрузке контекст запоминает снимок полей
- При flush сравнивает объект со снимком и сам формирует UPDATE
- По умолчанию в UPDATE все колонки: один готовый запрос на сущность. Только изменённые — `@DynamicUpdate`
- Без `@Transactional` контекст закрылся бы сразу после `findById`, и изменение потерялось бы
- Python: `Session` в SQLAlchemy — тот же unit of work

Spring Data JPA

ИТМО

```
public interface StudentRepository extends JpaRepository<Student, Long> {

  Optional<Student> findByEmail(String email);

  @Query("select s from Student s join fetch s.group where s.id = :id")
  Optional<Student> findWithGroup(long id);
}
```

- Реализацию никто не пишет: Spring генерирует **прокси** при старте
- `JpaRepository` даёт `save`, `findById`, `findAll(Pageable)`, `deleteById`
- Запрос по имени метода или явный JPQL в `@Query`
- Методы репозитория уже транзакционные

Запрос по имени метода

```
boolean existsByEmail(String email);
  select s1_0.id from students s1_0 where s1_0.email = ? fetch first ? rows only

long countByGroupName(String name);
  select count(s1_0.id) from students s1_0
  join study_groups g1_0 on g1_0.id = s1_0.group_id where g1_0.name = ?

List<Student> findByLastNameStartingWithIgnoreCase(String prefix);
  select ... from students s1_0 where upper(s1_0.last_name) like upper(?) escape '\'

List<Student> findTop10ByStatusOrderByLastNameAsc(Status status);
  select ... from students s1_0 where s1_0.status = ?
  order by s1_0.last_name fetch first ? rows only
```

Имя разбирается при старте: опечатка в поле — ошибка запуска.
`GroupName` — переход по связи, отсюда `join`

@Query: JPQL и SQL

JPQL — сущности и поля

```
@Query("""
    select s from Student s
    where s.group.name = :group
      and s.status = :status""")
List<Student> find(String group, Status status);
```

SQL — таблицы и колонки

```
@Query(nativeQuery = true, value = """
    select * from students
    where email ilike :pattern""")
List<Student> search(String pattern);
```

- JPQL переводится в SQL нужной БД; в нём нет таблиц и колонок, только классы и поля
- `nativeQuery` — когда нужна возможность конкретной СУБД: `ilike`, оконные функции, JSON
- Параметры `:group` передаются отдельно от текста запроса — защита от SQL-инъекций
- Изменяющий запрос: `@Modifying` + `@Query("update ...")`

Сортировка и пагинация

```
Page<Student> page = repo.findByStatus(Status.ACTIVE,
    PageRequest.of(2, 20, Sort.by("lastName")));      // page 3, 20 per page

select s1_0.id, s1_0.email, s1_0.group_id, s1_0.last_name, s1_0.status
from students s1_0
where s1_0.status = ?
order by s1_0.last_name
offset ? rows fetch first ? rows only                   -- offset 40, limit 20

select count(s1_0.id) from students s1_0 where s1_0.status = ?
```

`Page` содержит строки страницы и общее число записей: для этого Spring выполняет второй запрос с `count`.

Общее число не нужно — возвращайте `Slice`: Spring запросит на одну строку больше и узнает только, есть ли следующая страница.

Как увидеть SQL

application.properties

```
logging.level.org.hibernate.SQL=debug
logging.level.org.hibernate.orm.jdbc.bind=trace
spring.jpa.properties.hibernate.format_sql=true
```

в логе

```
DEBUG org.hibernate.SQL :
    select s1_0.id, s1_0.email, s1_0.group_id
    from students s1_0
    where s1_0.email=?
TRACE org.hibernate.orm.jdbc.bind :
    binding parameter (1:VARCHAR) <- [ivanov@itmo.ru]
```

- `org.hibernate.SQL` — текст запросов, `jdbc.bind` — значения параметров
- `spring.jpa.show-sql=true` пишет в `System.out` мимо логгера — лучше настроить уровень логирования
- Вместо таблиц в SQL Hibernate ставит псевдонимы: `s1_0` — первая таблица сущности `Student`
- Число запросов на один HTTP-запрос — первое, что стоит проверить

Проблема N+1

```
@Transactional(readOnly = true)
public List<GroupDto> groups() {
  return groupRepo.findAll().stream()
      .map(g -> new GroupDto(g.getName(),
                             g.getStudents().size()))
      .toList();
}
```

```
select g1_0.id, g1_0.name
from study_groups g1_0                          -- 1

select s1_0.id, s1_0.email, ...
from students s1_0 where s1_0.group_id = ?  -- N
select ... where s1_0.group_id = ?
select ... where s1_0.group_id = ?
...
```

- Один запрос за группами и ещё по одному на **каждую** группу: 1 + N
- 100 групп — 101 запрос, каждый с сетевой задержкой
- В коде этого не видно: `getStudents()` выглядит как обращение к полю
- EAGER не спасает: `findAll` студентов с EAGER-группой даёт те же N запросов за группами

Решение: `join fetch` и `@EntityGraph`

```
@Query("""
    select g from StudyGroup g
    left join fetch g.students""")
List<StudyGroup> findAllWithStudents();

@EntityGraph(attributePaths = "students")
List<StudyGroup> findByNameStartingWith(String p);
```

```
select g1_0.id, g1_0.name, s1_0.group_id,
       s1_0.id, s1_0.email, s1_0.last_name
from study_groups g1_0
left join students s1_0
  on g1_0.id = s1_0.group_id               -- 1 query
```

- Группы и студенты приходят одним запросом
- `@EntityGraph` — то же без JPQL: список связей для метода репозитория
- Hibernate 6 сам убирает повторы групп из результата, `distinct` не нужен
- Ловушка: `join fetch` коллекции + `Pageable` — Hibernate грузит **все** строки и режет страницу в памяти (предупреждение HHH90003004)

Решение: пакетная загрузка и DTO

Пакетная загрузка

```
spring.jpa.properties.hibernate:
  default_batch_fetch_size: 50

select g1_0.id, g1_0.name from study_groups g1_0
select ... from students s1_0
where s1_0.group_id = any (?)    -- 50 ids
```

Вместо N запросов — N/50. Работает с пагинацией. Для одной связи — `@BatchSize`.

Проекция в DTO

```
@Query("""
  select new ru.itmo.GroupDto(g.name, count(s))
  from StudyGroup g left join g.students s
  group by g.name""")
List<GroupDto> groupSizes();

select g1_0.name, count(s1_0.id)
from study_groups g1_0
left join students s1_0
  on g1_0.id = s1_0.group_id
group by g1_0.name
```

Нужны два поля — не грузить сущности вовсе: БД посчитает сама.

LazyInitializationException

Откуда берётся

Сервис загрузил студента и закрыл транзакцию. Контроллер обращается к `student.getGroup().getName()` — LAZY-прокси пытается сходить в БД, а контекста уже нет.

Что делать

Загружать нужные связи в запросе: `join fetch`, `@EntityGraph`.

Собирать DTO внутри сервиса, пока транзакция открыта.

Не делать связь EAGER «чтобы заработало».

Spring Boot по умолчанию держит контекст открытым до конца HTTP-запроса (`spring.jpa.open-in-view=true`): ошибки нет, но N+1 тихо уходит в контроллер

Итоги

Тема	Главное
пароли	медленный хеш с солью: bcrypt, Argon2id
JWT	подпись проверяется без хранилища, отзыв — через короткий срок и refresh
Spring Security	цепочка фильтров до MVC; 401 — кто вы, 403 — нельзя
HTTPS	ECDHE даёт ключ, сертификат и цепочка CA — подлинность сервера
сущности	`@Entity`, `@Id`, `@Column`, enum как `STRING`
связи	владелец — сторона с FK, загрузка LAZY
Spring Data	имя метода или `@Query` превращается в SQL
N+1	видно в логе SQL; `join fetch`, `@EntityGraph`, batch, DTO

Вопросы для самопроверки

1. Чем аутентификация отличается от авторизации? Какие коды им соответствуют?
2. Почему выданный JWT нельзя просто отозвать?
3. Почему 401 и 403 не доходят до `@RestControllerAdvice`?
4. Почему POST из Postman получает 403 при настройках Spring Security по умолчанию?
5. Зачем TLS асимметричное шифрование, если данные шифруются симметрично?
6. Как клиент проверяет, что сертификат выдан именно этому сайту?
7. Чем `IDENTITY` отличается от `SEQUENCE` для пакетной вставки?
8. Почему `EnumType.ORDINAL` опасен?
9. Какая сторона связи владеет внешним ключом?
10. Почему изменение сущности попадает в БД без `save`?
11. Откуда берётся N+1 и как его увидеть?
12. Почему `join fetch` коллекции плохо сочетается с пагинацией?

Источники

Безопасность

Spring Security Reference
OWASP Cheat Sheet Series: Password Storage, Authentication, JSON Web Token
RFC 7519 — JWT
RFC 8446 — TLS 1.3
RFC 5280 — сертификаты X.509

Данные

Spring Data JPA Reference
Hibernate ORM User Guide
Jakarta Persistence Specification
Vlad Mihalcea, «High-Performance Java Persistence»

Спасибо
за внимание!

ITMO more than a UNIVERSITY